

content-boundary

The content boundary: private material must not appear in a public repository

`scripts/verify-content-boundary.sh + .content-boundary-allow`

The rule, in one line

Never quote, paste, excerpt or transcribe content from a private repository into a file tracked by a public one. Naming a private *path* is fine. Copying what is *inside* it is not.

Why it is absolute

This umbrella spans both visibilities. A gitlink in a public repository exposes a commit SHA and nothing more, so material in a private submodule stays private — **as long as nobody copies it out**. One paragraph pasted into a public docs/ or specs/ file defeats that permanently: the text is then in a public repository's history, and history is not editable after a push. There is no undo, only disclosure plus a rewrite nobody can force other clones to take.

The pressure that produces this mistake is not carelessness, it is diligence. An agent told to document the work comprehensively reaches for an illustrative quote, and the quote is the leak.

The measured fleet (re-derive; do not trust this table)

Derived 2026-09-01 (afternoon) from .gitmodules plus this tree’s own remote, with visibility and push permission taken from the provider. **One row changed between the morning and the afternoon of the same day** — see the note under the table. That is the reason this heading says “do not trust this table”.

```
bash scripts/verify-content-boundary.sh --list-fleet
```

repository	role	evidence
. (the umbrella)	public	private=false, push=true
workshop	private	private=true
design-toolkit	public	private=false, push=true — was private=true this morning
ai_interviewing	private	private=true
monetization	private	private=true
milosvasic.ru	public	private=false, push=true
vasic.digital	public	private=false, push=true
submodules/ constitution	public	private=false, push=true
submodules/ containers	public	private=false, push=true
submodules/ LLMProvider	public	private=false, push=true
submodules/RAG	public	private=false, push=true
submodules/verdict	public	private=false, push=true
submodules/passage	public	private=false, push=true
submodules/superspec	out-of-scope	private=false, push=false — an

repository	role	evidence
		upstream we cannot push

Four of these (LLMProvider, RAG, verdict, passage) were added to .gitmodules by another agent *while this guard was being written*, and the run picked them up with no edit to any script. That is the point of deriving the fleet rather than listing it.

design-toolkit flipped from private to public between two runs on 2026-09-01, and the gate followed it with no edit to anything. The earlier revision of this table recorded it as private and was correct when written. The consequence is arithmetic, not cosmetic: everything design-toolkit contributed to the private corpus left it, so **no total measured before the flip is comparable to one measured after**. The incident note’s “377 → 232” and this document’s own older funnel both predate it. Re-derive; do not subtract.

This table records ONE remote per repository, and design-toolkit has two with DIFFERENT visibilities. The fleet is derived from submodule.<name>.url — a single URL per submodule — plus this tree’s own remote. design-toolkit’s row above says **public**, which is a measured fact about its GitHub origin vasic-digital/design-toolkit. Measured 2026-09-02, the same repository also has a GitLab mirror, git@gitlab.com:vasic-digital/design-toolkit.git, which is **private**:

```
gh api repos/vasic-digital/design-toolkit --
    jq .visibility # public
glab api "projects/vasic-digital%2Fdesign-toolkit" | jq
    -r .visibility # private
# and anonymously, which distinguishes "private" from "you have
# access":
curl -s -o /dev/null -w '%{http_code}\n' https://api.github.com/
    repos/vasic-digital/design-toolkit # 200
curl -s -o /dev/null -w '%{http_code}\n' https://gitlab.com/api/
    v4/projects/vasic-digital%2Fdesign-toolkit # 404
```

This does not change any boundary verdict, and the reason is measured rather than assumed. The mirror is 6 commits **behind** the public origin and its head is a strict **ancestor** of it, so every commit and every tag the mirror advertises is already reachable from the public GitHub head; its merge-request, issue, wiki, snippet, package and registry stores are all empty. The private side is a strict subset of the public side, so it contributes nothing to the private corpus and no

total above needs re-deriving on its account. Full measurement, the commands, and the honest boundary on what `ls-remote` can and cannot see: [constitution-adoption/REMOTE-ASYMMETRY.md §10](#).

What this IS evidence of: a single-remote blind spot shared by two gates. Neither this guard nor `scripts/verify-submodule-remote-sync.sh` can see a second remote for any submodule, because `.gitmodules` declares exactly one URL and neither reads a submodule's named remotes. Today that costs nothing here. It would cost something the moment a repository's mirror were *ahead* of, or divergent from, the URL in `.gitmodules` — a case no instrument in this tree can currently detect. Closing it needs a declared source of mirror URLs and is a separate operator decision, specified in §10.6 of that document. **Do not read this note as a reason to relax anything.**

There are three private submodules today, not four and not one. `workshop` holds the session recording and its notes and is the most sensitive; `ai_interviewing` and `monetization` are private too, and the same rule governs all three. The fleet is derived on every run, so a repository flipped from private to public — or a submodule added tomorrow — is picked up without an edit to any script.

What the guard actually detects

Three key spaces, all built in a **single pass** over each file, all compared across line boundaries. Text is folded to lowercase alphanumerics and everything else becomes a separator, so a quotation the pasting editor re-wrapped is still the same token run — which a line-oriented `grep` cannot see, and which matters, because the first real leak found in this tree was a quotation split across three lines.

class	key	catches
prose	normalised 10-token shingle	a copied paragraph or sentence
short	normalised 5-token shingle, private side restricted to the document's own heading / list-item / block-opening lines	a copied <i>heading list</i> — class D2 of the incident
name	a run of 2 or 3 consecutively capitalised tokens ,	a real person's full name — class D3

class	key	catches
	matched in either order	

The blind spot the first two classes were added to close

On 2026-09-01 the prose pass scored two public files **clean** while both reproduced a private notes document’s complete section-heading list, and while a third party’s full real name stood in three public locations. It was found by a human-directed audit. docs/content-boundary-incident-2026-09-01.md records this as findings **F2** and **F3**.

Three independent barriers stacked, and lowering any one of them alone would not have helped:

- 1. **The window.** Five of the seven headings are five or six tokens long. No shingle of them is ever built — not at 10, not at 8, not at 7.
- 2. **The 40-character floor.** A five-token phrase needs an average word length of 7.2 characters to clear it.
- 3. **Prose-shape, “≥ 2 distinct function words”.** A heading is a noun phrase. *“Nightly ingest of the partner catalogue”* has one.

Measured on a synthetic four-heading fixture built for the purpose (no real string in it), against the **unmodified** gate:

--window	verdict	matches
10 (the shipped default)	CLEAN — rc 0	0
8 (the value the incident note records)	CLEAN — rc 0	0
7	LEAK	1
6	LEAK	2
5	CLEAN — rc 0 again	0

Read the last row carefully. **“Just lower the window” does not work:** at 5 the gate goes blind a second time, for barriers 2 and 3 rather than barrier 1. Note also that the incident note records the window as **eight** and the shipped default was **ten** — the blind spot was one token wider than the incident report says.

Why it does not cry wolf

A naive line-grep on this tree produced **24,963 hits whose top thirty were all legitimate**. That number is the whole design problem: a gate reporting 24,963 findings is switched off within a week, and then the real leak ships. Seven filters run, six derived and one declared. Three of them (1b, 1c, 2b) exist only to keep the short and name classes affordable, and each states which class it touches — a filter that narrowed all three at once would have re-created the blind spot in a new place.

#	filter	applies to	derived?	measured effect on this tree
1	prose-shape — ≥ 2 distinct function words, ≥ 5 alphabetic tokens	prose	derived	removes CSS/ markup/ code boilerplate, the largest residual class
1b	prose-LINE — the private line must contain none of { } <>= \ ; ~ \$ @ # _ / and no] (	short, name	derived	–63% of the short class and –63% of the name class in one step: it removes the entire CSS / SCSS / HTML / Go / go . sum contribution
1c	structural — the private line must carry a heading/ bullet marker or open a block, and	short	derived	a short line <i>inside</i> a wrapped paragraph is ordinary prose the 10-token pass already covers

#	filter	applies to	derived?	measured effect on this tree
2	$be \leq --$ short-line-max (9) tokens path-reference — shingles built from the private repos' own <code>git ls-files</code> are subtracted, at each window in use name path-reference — a candidate whose every token already appears in the private source path or the public file's own path	prose, short	derived	clears the recording's filename wherever it is quoted, while leaving quotations of the recording's <i>notes</i> standing
2b	every token already appears in the private source path or the public file's own path	name	derived	35 matches; kills the “private deep-dive about product X → public product page for X” class
3	already-public — a key in ≥ 2 distinct public repositories is already public	all three, computed per class	derived	840 keys this run (prose 786, short 6, name 48)
4	declared — <code>.content-</code>	all three	declared	9 pairs, 4,196

#	filter	applies to	derived?	measured effect on this tree
	boundary-allow pairs			matches pardoned

Why the short pass does not resurrect the 24,963-hit flood

Because its precision does not come from the window. This was measured, not assumed. Dropping to a 5-token window with only the *old* filters produced **8,535** short-class rows on this tree — unusable, and exactly the outcome the naive line-grep was rejected for.

private-side restriction on short keys	short-class rows
none beyond “a source line of 5–12 tokens”	8,535
+ prose-LINE rule, + “line standing alone between two blank lines”	51
+ prose-LINE rule, + “heading marker or block-opening line”, ≤ 9 tokens	35 ← shipped

The middle row was **tried and rejected**, and it is recorded because the reason matters: “standing alone between two blank lines” is wrong, because in a real notes document a heading is followed *immediately* by its own body. It dropped the exact class it exists to catch. **Mutation M17b caught that before it shipped**, which is the whole argument for pairing a gate with a mutation.

The **public** side is deliberately left unrestricted. In the public file the same words may sit mid-paragraph, inside a bullet, or be re-wrapped by the editor that pasted them. Narrowing what may *become* a candidate is safe; narrowing where it may be *found* would be a new blind spot.

Distinctiveness weighting over rare word sequences was considered and **not used** for the short class: it was not needed once structure did the work, and it would have cost recall on headings built entirely from common words — which is what a heading usually is.

The name class, and how candidates are derived without writing a name down

No name is written into the script, into `.content-boundary-allow`, into the proof fixtures, or into this document. Writing one down would itself be the disclosure the check exists to prevent. Candidates are re-derived every run from the private repositories by two independent routes:

- **git identities** of the private repositories (`%an`, `%cn`, `user.name`). These are person names *by construction* — no heuristic is involved.
- **capitalised runs in private prose** whose every token is **rare in lowercase** across this corpus. That single test is what separates `Firstname Surname` from `Design Toolkit` or `Getting Started` without a gazetteer: a common noun that happens to be capitalised in a heading is abundant in lowercase elsewhere, and a surname is not.

The claim that the second route’s frequency rule was “scale-free — in a small fixture nothing clears the minimum count” is WITHDRAWN. Both halves were measurably false. See *The 2026-09-01 name-class tightening* below.

Route (a) is precise and was not touched. Route (b) is where all the noise was. Measured on 2026-09-01: of the 125 surviving name rows, route (a) contributed **zero** and route (b) contributed **every one**.

The repository owner’s own name is excluded — derived, not hardcoded. Names that are git identities of a **public** repository are subtracted, in *both* orders. 11 such forms were subtracted this run. The person who commits to the public repositories is publishing under that name already.

Matching is **order-insensitive**: `Surname Firstname` is checked as well as `Firstname Surname`, and because the comparison runs over a token stream, a name split across a line break is still found.

The name itself is never printed

The terminal output and the `--json` output both **withhold** the matched name and give a stable digest instead — `personal name #4f8d043f`. The `public file:line` is printed, which is what an operator needs to act. This is deliberate: gate output gets pasted into reports, CI logs and

incident notes, and that is one of the ways class D3 spread the first time. The digest lets a report say “*43 occurrences of 8 distinct names*” without any archive ever containing one.

Filter 2 is the one that answers the obvious objection. A public file may legitimately name `workshop/chapters/01/<recording>.mp4` in a build command; that is a reference, not a disclosure of what was said. Because the mask is built from the private index itself, it needs no maintenance.

Filter 4 uses **pairs** — one private glob, one public glob, one mandatory reason — so an exemption authorising design tokens to reach `design-system/` does not also authorise a transcript to reach it. A malformed row makes the run exit 2, never 0.

The stated limit of filter 3

“Already public in ≥ 2 repositories” cannot exonerate a passage leaked into two public repositories at once, and it is not claimed to. Every shingle it clears is counted and its private source files are named in the run’s own summary, so the exoneration is visible rather than silent.

The stated limit of filter 1

The model is **English prose**. Private *code* copied into a public repository is outside this instrument and is not claimed to be covered.

The 2026-09-01 name-class tightening

A triage adjudicated every one of the 125 name rows and found **125 of 125 rows and 20 of 20 distinct digests to be false positives — not one was a person**. A class that is 100% noise gets ignored, and an ignored detector is worse than no detector, because it looks like coverage. Three changes shipped; two candidates were rejected. Every figure below was re-derived on this tree, not inherited.

What was actually wrong: the tokeniser, not the filters

The N pass folded **every** non-alphanumeric run into a single separator. That is correct for the prose pass — it is what defeats re-wrapping — and wrong for names, because it **fuses list items into one “name”**: A/B/C became a single capitalised run. That run stands in neither text. It was a tokeniser defect, not a filter that was too loose.

1. Name-junction rule (shipped, both sides)

Two capitalised tokens form a candidate only when what stands between them is something a personal name may actually contain. It is written as an **allow-list** because that is the half that can be justified:

junction	verdict
whitespace only	name-compatible
a bare hyphen (Anne-Marie)	name-compatible
a comma with optional whitespace (Surname, Firstname)	name-compatible
anything else	a delimiter — break

At most **one** hyphen or comma; two is a break. So Alpha - Beta (spaced dash), Alpha--Beta, Alpha.Beta, Alpha/Beta, Alpha|Beta, Alpha;Beta, Alpha[Beta], **Alpha:** **Beta** and "Alpha",\n"Beta" all break, and so do the non-ASCII commas 、 and ٫.

It is applied to the PUBLIC side as well as the private one, against this script's usual doctrine that narrowing the public side creates a blind spot. The reason is that this is not a narrowing of where a name may be *found*: the fused run exists in neither corpus, and fixing the tokeniser on one side only would leave the two normalised differently — the one thing the script's own header says must never happen. Measured: applying it to the private side alone drops **0** of the 125 rows today; the public side is where every fusion sits.

Measured effect: 67 of 125 rows (54%), 4 of 20 digests.

2. No bridging a token-free line (shipped, both sides)

A capitalised run may be **re-wrapped onto the next line** — that is a real leak shape and mutation M19 holds it open — but it may not bridge a line with no tokens on it. A person is not written across a paragraph break, a horizontal rule, or a PDF page break.

Measured effect: 2 further rows, 2 further digests. (The triage reported 3 rows; re-derived here it is 2. The third row it counted crosses a plain line break inside a JSON array, not a blank line, and is cleared by rule 1 instead.)

3. Rank-independent frequency floor (shipped) — the one with a real cost

The old rule was `rank <= --name-rank (2000) AND count >= 5`. Two things are wrong with it and both are measured, not asserted:

- **An absolute rank is not scale-free.** Rank is a position in a *vocabulary*. On this tree rank 2000 lands at **count 89**, so every token of every surviving false positive passed straight through: they sit at **ranks 2167–14849**, and several occur **45 to 80** times. (The triage reported the corpus at 941,648 tokens and ranks 2072–12227; re-derived here the corpus is **1,325,163** private prose tokens and the range is **2167–14849**. Mine supersedes.)
- **A small fixture *does* clear the minimum count.** Mutation M22 needed exactly two planted lines; a third put the fixture's given name at 5 occurrences and the old rank rule swept it unaided.

The floor is now expressed **per million prose tokens** (`--name-ppm`, default **25**), so it tracks the corpus instead of a fixed position in it, and it is **0** in a small corpus, where nothing can clear it. On this tree it derives to **33 occurrences**. `--name-floor <n>` gives the absolute form, which the paired proof needs because no per-million rate reaches 1 in a fixture. The old rank rule is **kept and unioned** with it, so nothing that used to be ordinary stops being ordinary. **Route (a) is exempt from both**, deliberately.

Why 33 and not 20. The triage measured a floor of 20 clearing 13 of 20 digests — reproduced exactly here. It is still the wrong number, because the floor was chosen on the noise axis and the cost sits on the other one:

floor	digests left (of 20)	candidates lost (of the 445 the old build derived)
89 (today's effective rank cut)	20	0
40	12	221 (49.7%)
33 (shipped)	12	234 (52.6%)
30	11	245 (55.1%)
25	8	268 (60.2%)
20	7	289 (64.9%)

Below about 30 the floor stops sweeping ordinary English words and starts sweeping **low-frequency proper nouns** — qdrant (25), milvus (20), face (19), pinecone (15) — which is precisely the frequency class a person's name belongs to. Buying quiet there means buying it with exactly the recall the detector exists for. **33 is 3× the highest count among the tokens of the real private-only given names in this corpus (5 and 11).** The one real person-token above it (count 132) belongs to a *public* git identity and is subtracted by construction anyway.

The comma decision

The comma is allowed. Breaking on it would remove a further **15 rows (12%) and 4 more digests**, all comma-separated enumerations of product names. It was rejected anyway:

1. Surname, Firstname is a real personal-name rendering — the form an index, a bibliography and an attendance roster use — and the comma is the *only* punctuation it contains. Order-insensitive matching (mutation M15) exists to catch it; a comma break would delete the shape it matches.
2. A **series-scoped** exception (“break only inside a run of three or more comma-separated capitalised items”) was designed and rejected: it false-breaks the ordinary appositive Surname, Firstname, who chaired, which is exactly the leaked-roster prose it would exist to protect. And two of the four comma digests are *two-item* enumerations, which no structural rule can tell from Surname, Firstname.
3. The 15 rows are therefore a **declared, measured, unfixed** noise source.

Mutation M21 holds this decision open. If a later change makes the comma a hard separator to buy a quieter run, M21 goes red and says why.

Rejected: the intra-word hyphen

The triage’s own fourth, not-recommended proposal. Re-derived: it clears **7 rows and 2 digests** and costs **every double-barrelled given name**. Rejected.

The recall cost of each shipped change, added to the blind-spot list

change	what the gate can no longer see
junction rule	a name split across two markdown table cells (<code>\ </code> <code>Firstname \ Surname \ </code>), across two code spans, across two path segments, or written with a non-ASCII comma
token-free-line rule	a name whose two tokens sit on either side of a blank line, a horizontal rule, or a PDF page break
frequency floor	a person whose name token reaches 33 occurrences in the private corpus. This grows with the corpus : the two real given names here sit at 5 and 11, but a participant recurring across a longer course <i>will</i> cross it, and only route (a) — a git identity — still sees them, and only if they commit

Measured result

	rows	prose	short	name	distinct name digests	rc	runtime
before	367	207	35	125	20	1	206–294 s

	rows	prose	short	name	distinct name digests	rc	runtime
after	285	207	35	43	8	1	193–299 s

The derived candidate set went **445 → 163**, and the run now prints the floor it derived — 33 occurrence(s) over 1,325,231 prose tokens (25 per million, scale-free); 3,888 token(s) count as ordinary words — so it is never an invisible constant. (The corpus total moved 1,325,163 → 1,325,231 between the two runs; four other agents were editing this tree at the time. Every number here is a dated observation.)

The prose and short passes are provably untouched: comparing the two runs’ own `--json` output as sets, both are **byte-identical** — **207 and 35 rows, 0 added, 0 removed**. The name class lost **82 rows and 0 gained**.

Runtime is **within the noise** — see the paired table under *Cost*, where the two back-to-back samples disagree in sign. The first working build was reliably +60–70 s, because the junction signature was built eagerly for every line carrying two capitalised tokens. It is now built **lazily** — at most once per line, and only for a line the anchored name loop actually reaches. Fixing that also surfaced a real bug: an editing slip had dropped the public-side anchor test entirely, which changed no result (the join still filters by candidate) but did most of the damage.

The residue is still 100% false positives. 43 rows over 8 digests, dominated by comma-separated enumerations of product and technology names — the class the comma decision deliberately declines to sweep. That is recorded here rather than hidden, and **nothing was added to .content-boundary-allow**.

The stated limit of filter 2b (name path-reference)

It tests **every** token, never “any”, and that asymmetry is deliberate. The one artefact in this fleet whose *filename* carries a participant’s **first** name would otherwise pardon that person’s **full** name: first name in the path, surname not, so “all” keeps the finding and “any” would have lost it. The cost of “all” is the reverse case — a private file named after a person in full cannot be told apart from a public file naming that path.

The direction subtraction (shipped 2026-09-09) — the fifth filter

Every filter above answers **co-occurrence**: does this text stand on both sides of the boundary? None of them can answer **direction**.

Governance prose written in this public umbrella and propagated *into* a private submodule — which §11.4.157 lockstep and the private upstream-contributions/ staging tree make routine here — is indistinguishable, to filters 1 through 4, from private material leaking *out*. On this tree that ambiguity was most of the population: the offline probe recorded in `content-boundary-class-a-judgement.md` measured the largest structural class at **79.1 – 82.1 % public-first across five runs whose totals differed by more than 3,000 rows**. The totals moved; the direction did not.

That probe is now **inside the gate**, as a subtraction with the same shape and the same discipline as the four above it — derived every run, counted, and its recall cost printed.

The rule. A surviving row is subtracted only when its matched text is **provably public-first**: dated on **both** sides, with the earliest public commit **strictly** before the earliest private one. Equal timestamps are not public-first. Undated on either side is not public-first.

How a string is dated — text level, not file level. Every blob any commit of any repository on that side ever recorded is normalised with *the gate's own* rule (`[^A-Za-z0-9]+` → space, lowercased) and searched for the key as a token run. The date is the earliest commit whose blob **content** contains it, anywhere on that side — so a string that moved between files is still caught and a rename does not restart its clock. Blobs are walked oldest-first and a key leaves the search the moment it is dated: an exact pruning, since no later commit can date a string earlier.

Two classes are never touched, by two independent guards.

Class	Why it is excluded
name	The gate withholds a name's matched text by design. There is no string to date, and obtaining one would be the disclosure the redaction exists to prevent. The offline probe's own honest boundary says direction was

Class	Why it is excluded
fingerprint	never established for a single name row. Its value is withheld for the same reason, and its direction has never been measured on any population — the class was added on 2026-09-09, after the last probe. Subtracting an unmeasured class is how a gate goes quietly green.

Name and fingerprint keys are never handed to the dater, *and* the resolver refuses both classes by name whatever the key list contains. Mutations **M30d** and **M30e** plant a row of each with unambiguously public-first dates and require it to **survive**.

Recall cost, stated in full

1. **“Public committed first” is an order, not an exoneration.** If private material was pasted into a public file and **pushed** before the private document holding it was ever committed, this pass subtracts a real leak. The asymmetry is structural: the private corpus is read from the **working tree** and dated from **history**.
2. **Commit timestamps are author-controlled** and are rewritten by rebase, amend and filter-branch. This is not a tamper-evident clock.
3. **Only committed text can be dated.** A row whose text exists on one side in the working tree only is direction-**UNDETERMINED**: not subtracted, still reported, and counted in a named **COULD NOT DETERMINE** row.
4. **Only what history can be read as text is dated** — blobs at or below the corpus size cap that are not binary, mirroring the corpus the keys come from. A key that reached the corpus through **pdftotext** is therefore never dated (history holds the PDF, not its rendered text) and its row is **kept**.
5. It says nothing about whether a **kept** row is a disclosure. Removing four rows in five makes the fifth readable; it does not judge it.

Precedence is unchanged and still decided in one place: a leak outranks the direction-undetermined row like any other 2. --no-direction turns the pass off and reports the undirected population —

the paired proof uses it as the second half of every direction mutation (M30c), so a green case is attributable to the subtraction rather than to the fixture.

Dependency: python3. If it is absent the pass subtracts **nothing** and reports the whole eligible population as undetermined — a pass that cannot run is never reported as a pass that found nothing.

Exit codes

0 clean · 1 leak found, both sides named with a file:line on the public side · 2 **could not determine** — visibility unknown, no authenticated gh, a private submodule not initialised, a private PDF with no text extractor, or a malformed exemption file.

1 outranks 2: a leak is a fact. Rows that could not be checked are printed on **every** run whatever the exit code, so a 1 never hides a 2 and a 2 never reads as a 0. **A repository whose visibility cannot be established is state 2, never a pass** — “I could not look” is not “clean”.

Cost, declared rather than discovered

Runtime is dominated by page-cache state, not by the filters, and any single number here would be misleading. Measured on this tree on 2026-09-01, all six runs on the same host:

build	runs	wall clock	peak RSS
before (prose only)	2	175 s, 234 s	193 MB
prose + short + name	4	124 s, 210 s, 238 s, 253 s	194 MB
+ the 2026-09-01 name tightening (shipped build)	4	193 s, 209 s, 269 s, 299 s	194 MB
(two discarded intermediate builds, kept for the record)	2	265 s, 277 s	—

The only fair comparison is the **paired** one — the two runs made back to back against the same tree: 175 s → 210 s, +20%.

The 2026-09-01 name tightening has no measurable runtime cost, and the honest form of that statement is that the two paired samples disagree in sign:

paired sample, back to back, same host state	before	after
1	259.5 s	298.7 s (+15%)
2	294.2 s	208.5 s (-29%)

Unpaired observations span **206–294 s before** and **193–299 s after**. A single number in either direction would be a bluff. What is measurable is that the first working build was reliably worse — 265 s and 277 s — because the junction signature was built eagerly for every line carrying two capitalised tokens, and because an editing slip had dropped the public-side anchor test. The signature is now built lazily and the anchor is restored; those two rows are kept in the table because a regression nobody records is a regression that returns.

The fastest observed *new* run (124 s) is faster than the fastest observed *old* run, because by then the tree was in cache and the host was idle.
Peak memory is unchanged.

The added work is one pass, not three: the emitter builds all three key spaces while it already has the file open, and the public side is *anchored* on the private side’s own first tokens, so a short shingle or a name run is built only where the private corpus has something to compare it to. Private key spaces this run: **800,131** long keys, **2,838** short keys, and **163** name candidates (**445** before the 2026-09-01 tightening).

The earlier **~370 s** figure in this document is **withdrawn, not restated**: it was measured before design-toolkit became public and left the private corpus, so it does not describe the same computation.

CB_TRACE=1 prints a per-stage progress trace; minutes of silence are indistinguishable from a hang.

This is **not** a pre-push-hook cost. It belongs in a pre-tag / release sweep or a deliberate audit, the same trade docs/check-registry.md records for --run-proofs at 243 s. It is **not currently wired into scripts/pre-push-gates.sh** — that file was under concurrent edit and wiring it is the controller's call.

The measured funnel on this tree

Measured 2026-09-01, afternoon, **after** design-toolkit became public and **while** a second redaction wave was landing in the working tree. Totals move between runs for both reasons; the per-class split is the part worth reading.

stage	rows
naive line-grep, for comparison (dated: pre-flip)	24,963
prose — after all filters	207
short — after all filters (5 distinct phrases, amplified by the 16- language _content_* copies)	35
name — after all filters, after the 2026-09-01 tightening (8 distinct digests)	43
total, rc 1	285

The name row read **125 rows / 20 digests** before the tightening described above. A triage adjudicated all 125 and found **every one** a false positive; the tightening removed 82 of them without touching the prose or short passes.

The name class is a review queue, not a verdict, and the 43 that remain are *also* all false positives — dominated by comma-separated enumerations of product and technology names, which the comma decision deliberately declines to sweep.

None of these has been adjudicated *into* the exemption file, and **none has been added to .content-boundary-allow**. Silencing a finding this instrument has not been told is benign is the exact failure it exists to prevent; the pairs file says so itself.

Paired proof

```
bash scripts/verify-content-boundary.sh --prove-failure
```

Re-measured 2026-09-01 after the name-class tightening: **31 assertions passed, 0 failed, 22 mutations run, rc 0, 175 s.** (The earlier **23 / 17 / 117 s** figure described the battery before M18–M22 were added.)

Twenty-two mutations against a **synthetic** fixture that is green by construction. The live tree is a *reported* pre-flight that cannot disable the battery — the shape this repository had to adopt after two gates were found whose proofs baselined against the live tree and therefore ran zero mutations the moment the tree had a real finding. That failure mode is not hypothetical here: **this gate is red on this tree today**, so a live-tree control would have been inoperative from its first run.

Every fixture string is synthetic and was written for the proof. No heading, sentence or name from any real private document appears in it, and the two personal names are invented token pairs belonging to nobody. A fixture carrying the real leaked content would re-leak it into this public repository — the proof would become the incident.

case	seeded mutation	expected	got
C0	synthetic control, nothing planted	rc 0	rc 0
M1	verbatim quote from a private file	rc 1 + both sides named	rc 1
M2	the same quote re-wrapped onto one line	rc 1	rc 1
M3	a path <i>reference</i> only	rc 0	rc 0
M4	an exemption must not pardon other content	rc 1	rc 1
M5	uninitialised private submodule	rc 2	rc 2

case	seeded mutation	expected	got
M6	a leak outranks an undetermined row	rc 1	rc 1
M7	malformed exemption file	rc 2	rc 2
M8	provider client unusable, against the real tree	rc 2	rc 2
M9	--fleet-spec pointed at the real repository	rc 2 + refusal	rc 2 + refusal
M11	a five-token private heading, reproduced verbatim	rc 1	rc 1
M11b	the same tree with --no- short	rc 0 — the defect	rc 0
M12	a six-token private heading	rc 1	rc 1
M13	a heading re- wrapped across a line break	rc 1	rc 1
M14	a personal name, forward order	rc 1	rc 1
M14b	the name is withheld from the output	withheld	withheld
M15	a personal name, reversed order	rc 1	rc 1
M16	CONTROL — near-miss	rc 0	rc 0

case	seeded mutation	expected	got
M17a	prose, a capitalised product-style pair, and the private notes' own vocabulary recombined regression: a private notes doc's heading list reproduced in a public file, --no-short	rc 0 — this is the original leak	rc 0
M17b	the same tree, short pass enabled	rc 1	rc 1
M18a	CONTROL — the same two name tokens fused by /, \, ;, ., **: ** and ", \n"	rc 0	rc 0
M18b	the same tokens, separated by a space	rc 1	rc 1
M19	a name re-wrapped across a plain line break	rc 1	rc 1
M20a	CONTROL — a name split across a blank line	rc 0	rc 0
M20b	the same two lines, made adjacent	rc 1	rc 1
M21		rc 1	rc 1

case	seeded mutation	expected	got
M22a	Surname, Firstname — the comma decision, held open a planted name with the frequency floor disabled	rc 1	rc 1
M22b	the same tree with - -name- floor 3 — the declared recall cost	rc 0	rc 0
M10	the control is still clean after every mutation	rc 0	rc 0

M11b and M17a are the load-bearing halves. Without them the battery would prove only that the tree can be made red, not that **the short pass is what makes it red** — and M17a is the incident itself, reconstructed from synthetic parts: a notes document whose short headings are separated by their own body text, and a public file that reproduces the *heading list* and quotes no sentence at all.

M18b, M19, M20b and M22a are the load-bearing halves of the 2026-09-01 tightening. Each pairs the shape a rule now *clears* with a shape built from the **same two tokens** that it must still catch — forward (M14), reversed (M15), comma-reversed (M21), re-wrapped (M19), spaced (M18b), adjacent-line (M20b). Without those halves, every new CONTROL would also pass if the name pass had simply stopped working.

M22 is the pair that exhibits a **cost** rather than a win: the same tree, the same planted name, red with the floor off and green with it on. A tightening whose recall cost is not demonstrable is a tightening nobody can audit.

M16 is the control that keeps the answer honest. It plants ordinary prose, a Capitalised Pair whose tokens are common lowercase words in the private corpus, and a five-token phrase built from the private notes' own vocabulary but never written in them. All three stay clean, so the gate is demonstrably not matching everything.

Registered in scripts/check-registry.tsv as content-boundary, so scripts/verify-check-registry.sh fails if it goes missing. Re-measured 2026-09-01 after the short and name passes landed: **25 PASS / 0 FAIL / 5 DEBT / 0 UNDET, rc 0**, R5 green. The registry row is unchanged — same entry point, same --prove-failure proof, same --root / nonexistent → rc 2 probe (verified).

Status of the incident's open findings

finding	before	now
F2 — the gate cannot see class D2 (headings)	open	closed by the short class ; M11, M12, M13, M17 are the proof, M11b and M17a are the paired demonstration of the original defect
F3 — the gate cannot detect a personal name at all	open	a detector now exists ; candidates are derived, never listed, and the name is never printed. It is not a solved problem — see the blind spots below — but “cannot detect at all” is no longer true

The incident note proposed closing F3 with “*a privately-held name deny-list, read from outside the tracked tree*”. That was **not** the route taken: a deny-list is a file someone has to write a real name into and then keep synchronised, and it fails silently the moment a name is missing from it. Deriving candidates from the private repositories themselves needs no list, no maintenance, and no name written down anywhere.

What this instrument still cannot see — declared, not implied away

Every instrument in this tree declares its blind spots. These are the new ones and the surviving old ones, stated plainly rather than left for the next incident to discover:

- **A 5-to-9 token fragment lifted from the MIDDLE of a long private prose line.** The short pass indexes structural lines only; the long pass needs 10 tokens. Fragments in that band are covered by neither. This is the price paid for not resurrecting the 8,535-row flood, and it is a real gap.
- **Non-ASCII text of any kind.** Tokenisation is [[^]A-Za-z0-9]. Cyrillic, Greek and CJK private material is invisible to *every* pass, and so is a name written in one of them. This predates the short and name passes and is not fixed by them. It also means a repository owner whose git identity is recorded in Cyrillic is not subtracted from the name candidates by that route.
- **A name that is not Aa+ Aa+:** lowercase, ALL CAPS, McDonald, O'Brien, a mononym.
- **A name whose tokens are frequent enough in the private corpus** to enter the --name-rank common set **or to reach the --name-ppm frequency floor.** The floor is the sharper of the two and **it grows with the corpus:** 33 occurrences today, against real private-only given names sitting at 5 and 11. A participant recurring across a longer course will cross it, and only a git identity still sees them. *(Added 2026-09-01 — the cost of tightening 3.)*
- **A name whose two tokens are separated by anything other than whitespace, one hyphen, or one comma** — including a name split across two markdown **table cells**, two code spans or two path segments, and any name written with a non-ASCII comma (、, ,). *(Added 2026-09-01 — the cost of tightening 1.)*
- **A name whose two tokens sit on either side of a token-free line** — a paragraph break, a horizontal rule, or a PDF page break. A name re-wrapped onto the immediately following line is still seen (mutation M19). *(Added 2026-09-01 — the cost of tightening 2.)*
- **A name present in ≥ 2 public repositories**, which the already-public rule exonerates by design.
- **Private CODE copied into a public repository.** Filter 1 models English prose; filter 1b now explicitly *excludes* code lines from the two new classes.
- **Translated, paraphrased or reordered content.** This is a verbatim matcher.

- **Anything in git HISTORY.** Every pass reads the working tree only. A green run means the tip of the tree is clean and says nothing whatever about the commits behind it — which, for the 2026-09-01 incident, is the part that still matters.
- **A private repository that is private at the provider but whose content this process cannot read** is state 2, never 0.

Current state: this gate is RED, and that is the correct reading

`bash scripts/verify-content-boundary.sh` is **rc 1** today: **285 surviving rows — prose 207, short 35, name 43**, 0 undetermined. Nothing in that number has been silenced to make it smaller, and `.content-boundary-allow` was not touched by the 2026-09-01 tightening.

The groups are left visible for the same reason as before — a human decides, not a glob: `ai_interviewing` project deep-dives overlapping this repository's own site copy, the workshop's governance carriers quoted into `specs/001-workshop-curriculum-platform/quickstart.md`, and the 8 distinct name digests that remain.

A red gate is not this gate failing. It is this gate working on a tree that has not been remediated. `docs/content-boundary-incident-2026-09-01.md` is the record of what has and has not been decided.

Bugs these mutations caught, recorded so they are not reintroduced

M5 — a false CLEAN from an uninitialised submodule.

`git -C <dir> rev-parse --git-dir` **succeeds inside an empty directory** that merely sits within another repository: it walks up and answers about the parent. An uninitialised submodule is exactly such a directory, so the obvious check reported it as healthy and its content silently counted as “nothing to find” — a false CLEAN, the one verdict this instrument must never produce by accident. `is_repo_root()` now requires the directory to *be* the top level git resolves for it. Mutation M5 is what found this; without the paired proof it would have shipped.

M17b — a structural rule that dropped the class it was written for.

The short pass's first cut required a private heading to stand **alone between two blank lines**. It looked right, it cut the false positives from

8,535 to 51 — and it scored the reconstructed incident **clean**, because in a real notes document a heading is followed immediately by its own body. The rule is now “carries a heading/bullet marker **or** opens a block”. The mutation caught it in the same session it was written.

M22 — a “scale-free” claim that a fixture disproved in one line. This document and the script both said the name-frequency rule was scale-free and that “in a small fixture nothing clears the minimum count”. Building M22 showed otherwise immediately: three planted private lines put the fixture’s given name at 5 occurrences, which is exactly NAME_MINCOUNT, and the old rank rule swept the candidate unaided — so *both* halves of the pair went green and the mutation proved nothing about the floor it was written for. M22 now plants exactly two lines, and the comment says why. The claim is withdrawn in both files rather than quietly edited.

A dropped anchor test, found by a runtime regression. While adding the junction rule, an edit removed `if (useNA && !(tok[i] in NA)) continue` from the name loop. It changed **no result** — the hit join still filters by candidate — so no assertion caught it. It showed up as the gate going from 206 s to 277 s. A performance measurement was the only thing standing between that and shipping.

A counting bug in this gate’s own summary. The per-class totals were taken from the three pre-deduplication files while the reported total came from the `sort -u`’d union, so the parts did not add up to the whole (129 vs 125). They are now all counted from the deduplicated result. A summary whose parts do not add up to its own total is the kind of small dishonesty that makes a whole report unusable.